Postupak HLS-a

Prilikom pokretanja HLS analize prvi put, iskorišćenost DSP blokova prelazila je 200%, pa je originalna verzija, u kojoj se radilo obrađivanje u batch-evima od po 5 sample-ova, zamenjena novom verzijom koda, u kojoj se obrađuje sample-po-sample.

Ponovno pokretanje HLS analize nije odmah dalo dobre rezultat, pa se uz par izmena, iskorišćenost DSP blokova i LUT-ova smanjila.
Problem je bio što HLS instancira fast_inv_sqrt dva puta kao posebne hardverske blokove. Možemo ga prisiliti da deli jednu hardversku instancu za oba poziva:
DSP usage od fast_inv_sqrt se prepolovljava jer oba poziva dele isti HW blok.

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	940.000	95	no	function	0	95	8,582	18,388	0
fast_inv_sqrt	200.000	20	no	function	0	5	560	917	0

Slika 1. Rezultati HLS-a u Vitisu

Koristimo 95 DSP blokova, a na raspolaganju imamo samo 80. Pored toga, broj LUT-ova takođe prelazi granicu.
Nakon toga, iskorišćenost resursa još uvek nije bila zadovoljavajuća, pa je odlučeno da se pokuša sa smanjenjem broja bitova samo za intermedijarne tipove:

- dot_t: 26→18 bita – koristi se samo za poređenje (if dot < 0), ne za dalje računanje
- magsq_t: 22→18 bita – ulaz u fast_inv_sqrt, malo utiče na krajnji rezultat
- invmag_t: 24→18 bita – intermedijaran, rezultat se odmah kasti na norm_t

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	940.000	95	no	function	0	92	8,536	18,200	0
fast_inv_sqrt	200.000	20	no	function	0	5	560	917	0

Slika 2. Rezultati HLS-a u Vitisu nakon smanjenja broja bitova za određene tipove

Obzirom da rezultati pokazuju da je iskorišćenost još uvek velika, uvode se određene pragme koje bi trebalo da pomognu. Prva pragma koja je uvedena jeste ALLOCATION operation.

ALLOCATION operation ograničava ukupan broj množilica i primorava HLS da ih time-multipleksira kroz manje DSP instanci.
Ključna nova pragma je:

```
#pragma HLS ALLOCATION operation instances=mul limit=30
```

Ovo primorava HLS da sve množenja rasporedi kroz 30 DSP instanci umesto ~90 paralelnih.

```
+ Performance & Resource Estimates:

    PS: '+' for module; 'o' for loop; '*' for dataflow
    +------------------+------+------+---------+---------+----------+---------+------+----------+------+-----------+-----------+--------------+-----+
    |      Modules     | Issue|      |      Latency      | Iteration|         | Trip |          |      |           |           |              |     |
    |      & Loops     | Type | Slack| (cycles)|   (ns)  |  Latency | Interval| Count| Pipelined| BRAM |    DSP    |     FF    |      LUT     | URAM|
    +------------------+------+------+---------+---------+----------+---------+------+----------+------+-----------+-----------+--------------+-----+
    |+ ahrs_ip_hls     |     -|  0.04|       94|  940.000|         -|       95|     -|        no|     -|  86 (107%)|  7678 (21%)|  18453 (104%)|    -|
    | + fast_inv_sqrt  |     -|  0.04|       20|  200.000|         -|       20|     -|        no|     -|     5 (6%)|   560 (1%)|    917 (5%)|    -|
    +------------------+------+------+---------+---------+----------+---------+------+----------+------+-----------+-----------+--------------+-----+
```

Slika 3. Rezultati HLS-a u Vitisu nakon uvođenja ALLOCATION operation pragme

Rezultati još uvek nisu bili sasvim odgovarajući, pa je sledeća ideja bila da se poveća clk sa 10ns na 15ns, HLS time dobija više slack-a i može da ulančava operacije kroz više ciklusa umesto da instancira paralelne DSP kopije.

Timing Estimate

TARGET	ESTIMATED	UNCERTAINTY
15.00 ns	10.533 ns	4.05 ns

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	975.000	66	no	function	0	86	5,310	18,245	0
fast_inv_sqrt	225.000	15	no	function	0	5	562	897	0

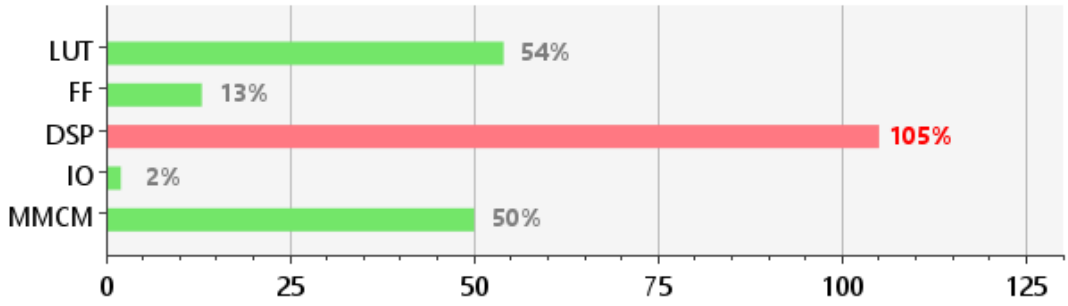
Slika 4. Rezultati HLS-a u Vitisu nakon promene clk na 15ns

Vivado implementacija ponekad daje bolje rezultate od HLS procene jer radi fizičko mapiranje na čip. HLS procena je konzervativna. Postoji šansa da čak i sa 86 DSP procenom, Vivado implementacija stvarno stane u 80 DSP. Zbog toga je sledeći korak bio da se eksportuje IP iz HLS-a i da se uvuče u Vivado projekat, pa da se tamo odradi sinteza. Korišćen je identičan kod, kao kod na osnovu kog su dobijeni rezultati sa slike 4.

Name	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	DSPs (80)	Bonded IOB (100)	BUFGCTRL (32)	MMCME2_ADV (2)
ahrs_bd	9557	4417	30	84	2	2	1
ahrs_ip_hls_0 (ahrs_bd_ahrs_ip_hls_0_0)	9538	4377	30	84	0	0	0

Slika 5. Rezultati sinteze u Vivadu

Resource	Utilization	Available	Utilization %
LUT	9557	17600	54.30
FF	4417	35200	12.55
DSP	84	80	105.00
IO	2	100	2.00
MMCM	1	2	50.00



Slika 6. Grafički prikaz rezultata na osnovu sinteze odrađene u Vivadu

Broj DSP blokova se smanjio, ali nedovoljno, a broj LUT-ova je čak značajno manji, nego što je bio prilikom pokretanja HLS-a u Vitisu. Zbog toga će, u daljem radu, biti nastavljeno pokretanje i sinteze u Vivadu, kako bi se došlo do što realističnijih rezultata.

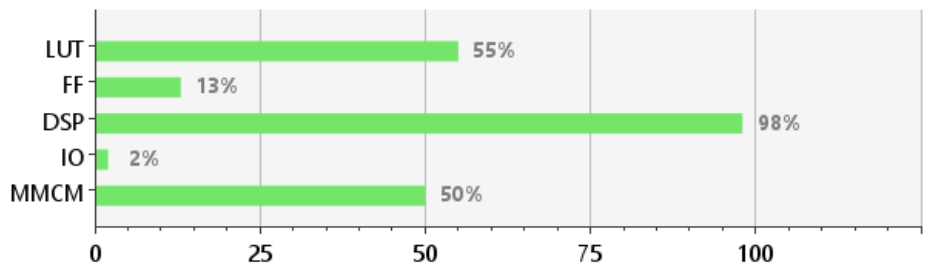
Rezultati su bili jako blizu granice, te je pokušano smanjenje limita u pragmi sa 30 na 26:

```
#pragma HLS ALLOCATION operation instances=mul limit=26
```

Name	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	F8 Muxes (4400)	DSPs (80)	Bonded IOB (100)	BUFGCTRL (32)	MMCME2_ADV (2)
ahrs_bd_wrapper	9757	4495	31	1	78	2	2	1
ahrs_bd_i (ahrs_bd)	9757	4495	31	1	78	0	2	1

Slika 7. Rezultati sinteze u Vivadu nakon smanjenja limita pragme

Resource	Utilization	Available	Utilization %
LUT	9757	17600	55.44
FF	4495	35200	12.77
DSP	78	80	97.50
IO	2	100	2.00
MMCM	1	2	50.00



Slika 8. Grafički prikaz rezultata na osnovu sinteze odrađene u Vivadu nakon smanjenja limita pragme

Nakon dobijanja zadovoljavajućih rezultata, pokrenut je i HLS u Vitisu, kako bi se dobijene vrednosti potvrdile.

Estimated Quality of Results

Timing Estimate

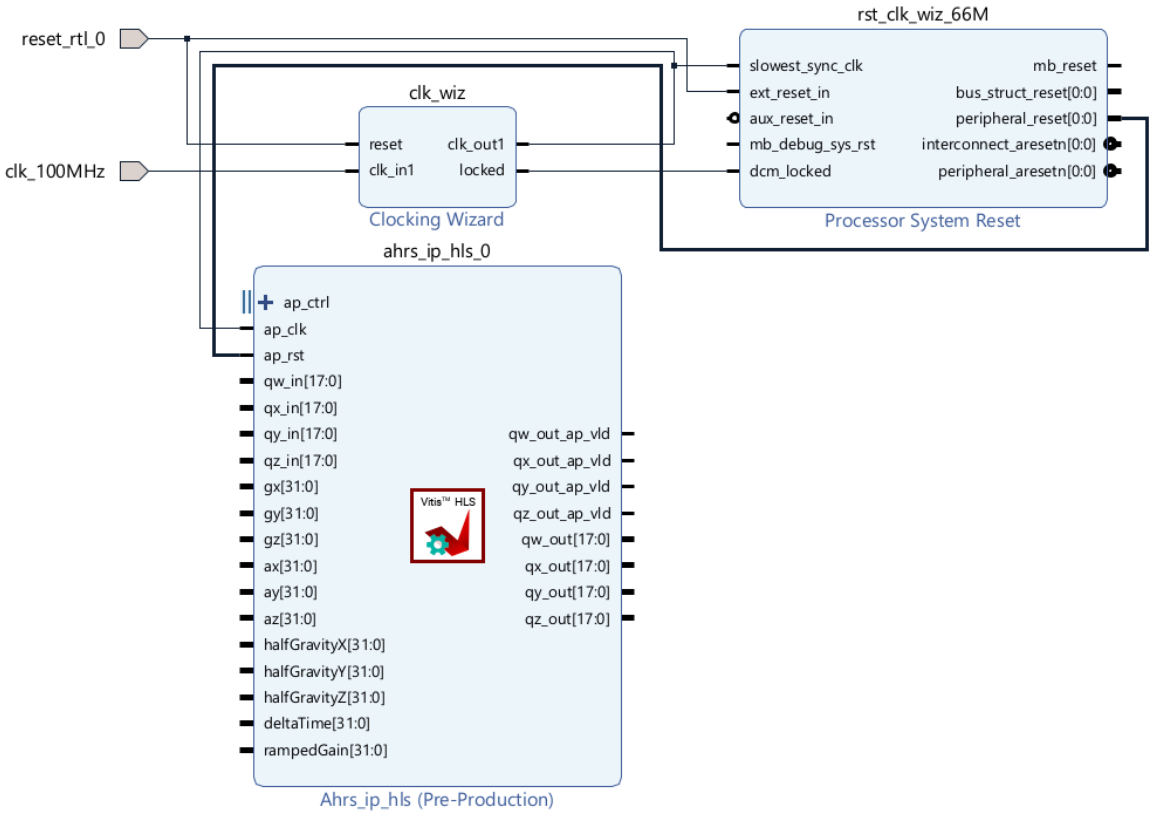
TARGET	ESTIMATED	UNCERTAINTY
15.00 ns	10.533 ns	4.05 ns

Performance & Resource Estimates

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
ahrs_ip_hls (1)	975.000	66	no	function	0	78	5,320	18,158	0
fast_inv_sqrt	225.000	15	no	function	0	5	562	897	0

Slika 9. Pokretanje HLS-a u Vitisu



Slika 10. Izgled IP bloka u Vivadu

Rezultat je uspešno dokazao da je broj iskorišćenih DSP blokova u granicama u odnosu na dostupne resurse. Iskorišćenost DSP blokova spala je na 97%, dok je broj iskorišćenih LUT-ova, u odnosu na dostupan broj, jednak 55%.

Bez pragme, HLS instancira sve množilice paralelno – svako množenje dobija svoj DSP48E1 blok koji radi istovremeno sa svim ostalim. To je brže ali troši ~92 DSP-a odjednom.

Sa limit=26, HLS sme da instancira maksimalno 26 množača u isto vreme. Preostalih množenja se raspoređuju sekvencijalno kroz tih 26 instanci – jedno čeka da drugo završi pa koristi isti DSP blok.

Preciznost – nula uticaja. Pragma ne menja ni jedan bit ni jednu operaciju. Isti izračun, isti redosled, isti tipovi. Samo se menja *kada* se koji množilic aktivira, ne *šta* računa.

Brzina – minimalan uticaj. Latencija po uzorku je porasla sa 940ns na 975ns – razlika od 35ns. Na 100Hz mo10ms između uzoraka, a IP završi za ispod 1ms.